



SECR2043 OPERATING SYSTEM

**TITLE: SIMULATION OF FIXED AND DYNAMIC MEMORY
MANAGEMENT USING NEXT FIT AND WORST FIT ALGORITHMS**

LECTURER: DR. DANLAMI GABI

SECTION: 03

GROUP NAME: FOUR PEOPLE

GROUP MEMBER	MATRICES NO.
JOANNE CHING YIN XUAN	A23CS0227
LIM YU HAN	A23CS0241
CHOH JING YI	A23CS0296
NEO LI XIN	A23CS0253

Table Of Content

1.0 Abstract.....	3
2.0 Introduction.....	3
3.0 Problem Statement.....	4
4.0 Related Works	5
5.0 Methodology	5
5.1 Project Management Flowchart	6
5.2 Understanding What to Simulate	7
5.3 Hardware and Software Requirements.....	7
5.4 Task Distribution	7
6.0 Design & Implementation:	8
6.1 System Flowchart	8
6.2 Code Implementation.....	9
7.0 Results and Discussion.....	13
7.1 Interface and Execution.....	13
7.2 Output Analysis	13
7.3 Discussion	14
8.0 Future Work & Conclusion.....	14
8.1 Future Work	14
8.2 Conclusion.....	14
9.0 Acknowledgement	15
10.0 References	15
11.0 Presentation Link	15

1.0 Abstract

Operating systems rely heavily on memory management to allocate memory efficiently to processes. This project creates a Java simulation to demonstrate fixed and dynamic memory partitioning with Next Fit and Worst Fit algorithms. The simulation mimics how a computer system allocates memory to running applications. Users specify memory block and process sizes, and the application assigns processes to blocks and displays the results. The research examines algorithm performance and identifies concerns such as internal and external fragmentation. Results indicate that Next Fit is faster but may ignore ideal blocks. Worst Fit decreases fragmentation in Dynamic partitioning but leaves large unused blocks in Fixed partitioning. The simulation offers an interactive tool for learning memory management topics.

2.0 Introduction

Memory is a limited and crucial resource in modern computer systems. The Operating System (OS) manages not only the CPU and I/O devices but also allocates memory efficiently to ongoing programs. Poor memory management can lead to low system performance and inefficient resource utilization. Memory management strategies are used by operating systems to improve performance and ensure that processes are properly allocated to memory blocks. This research focuses on two memory partitioning approaches, Fixed Partitioning and Dynamic Partitioning, and incorporates two well-known memory allocation algorithms, Next Fit and Worst Fit, into these partitioning methods. The project is written in Java and seeks to simulate memory allocation based on user inputs. It assists students in understanding the internal workings of operating system memory management by displaying how different algorithms react under a variety of inputs and settings.

3.0 Problem Statement

Memory management algorithms are often hard to understand, especially for students and beginners learning about operating systems. These algorithms involve ideas like how memory is given to processes, how unused memory gets wasted which called fragmentation, and how the system picks memory blocks for each process. These ideas can be confusing without a clear way to see how they work. Traditional ways of teaching like lectures, textbooks don't always help students really understand what happens during memory allocation. Because of this, many learners find it hard to connect the theory with how it works in real systems.

One big problem is that it's hard to show how memory is allocated step by step in real time. Things like internal and external fragmentation, memory allocation failures, and how memory blocks are chosen are difficult to picture without a tool that shows them clearly. Without seeing these processes in action, students may only get a basic understanding and struggle when trying to apply it in programming or system design.

To solve this problem, this project creates an interactive learning tool using Java. The tool lets users enter memory block sizes and process sizes, then choose between Fixed and Dynamic Partitioning. Users can also select either the Next Fit or Worst Fit algorithm. The simulation then shows how memory is used, what parts are wasted, and which processes get rejected.

By letting users compare both algorithms in different partitioning types, the tool helps explain the pros and cons of each method. Some methods work better in certain situations, while others might leave more wasted space. This project helps students better understand memory management by showing how these algorithms work in a simple and clear way. It makes learning easier, more visual, and more engaging.

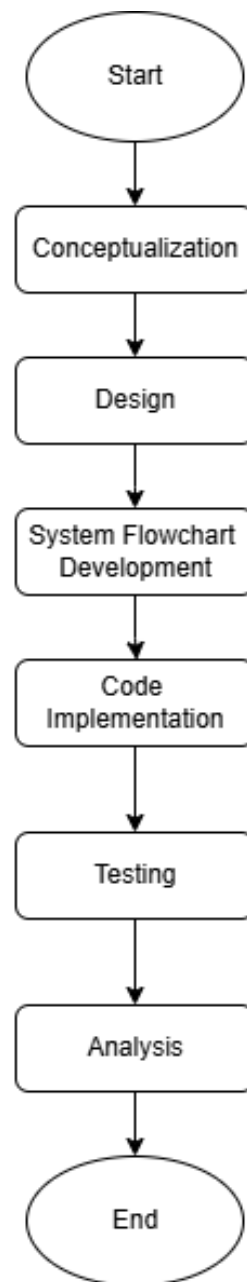
4.0 Related Works

Memory management has been extensively studied in operating system literature. Silberschatz et al. (2018) describe Fixed and Dynamic partitioning, noting that Fixed partitioning is simple but prone to internal fragmentation, while Dynamic partitioning reduces waste but requires compaction to manage external fragmentation. Tanenbaum (2014) discusses allocation algorithms, highlighting Next Fit's simplicity and Worst Fit's attempt to minimize fragmentation. Previous simulations, such as those by Stallings (2017), provide theoretical models but lack interactive visualizations. Online tools like OS simulators (OS Simulator, 2023) allow users to input block and process sizes but do not support dynamic resizing or detailed fragmentation analysis. Our simulation enhances these efforts by integrating both memory strategies and algorithms, focusing on clarity and interaction, while being simple and accessible through Java's console interface.

5.0 Methodology

This project uses a simulation-based approach to help users understand how memory is allocated in operating systems. The idea is to let users interact with a simple Java program that shows how different algorithms work when assigning processes to memory. The whole development process can be broken down into five main steps: conceptualization, design, implementation, testing, and analysis.

5.1 Project Management Flowchart



5.2 Understanding What to Simulate

Steps in the Flowchart:

1. Conceptualization: Define project goals and objectives.
2. Design: Develop the system flowchart of how memory is allocated in operating systems.
3. Implementation: Code the simulation using Visual Studio Code.
4. Testing: Validate the simulation to ensure it functions as expected.
5. Analysis: Evaluate the results of the simulation to measure performance improvements.

5.3 Hardware and Software Requirements

Software: Visual Studio Code for coding.

Hardware: Standard computer

5.4 Task Distribution

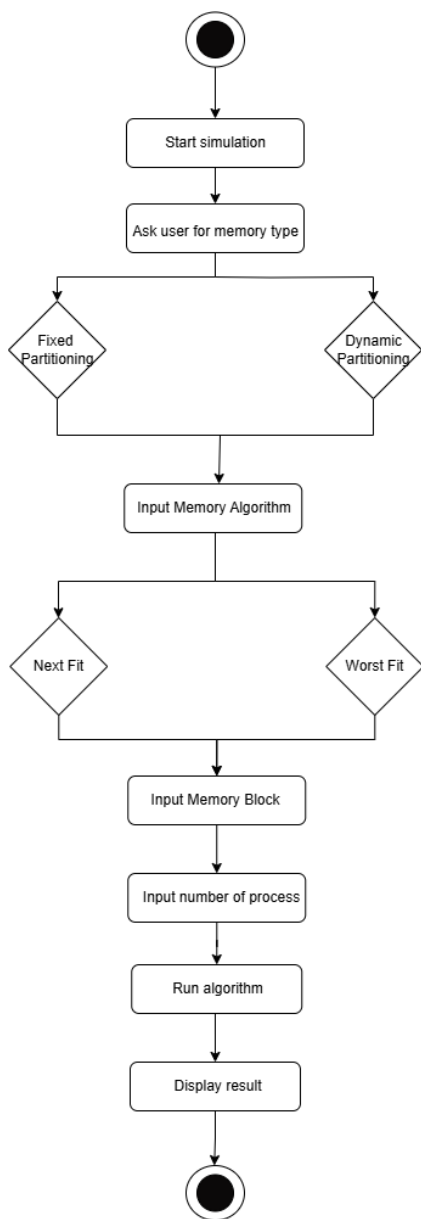
- Literature Review: Choh Jing Yi & Neo Li Xin & Joanne Ching Yin Xuan & Lim Yu Han
- Simulation Design and Coding: Joanne Ching Yin Xuan & Lim Yu Han
- Testing and Analysis: Choh Jing Yi & Neo Li Xin
- Report Writing: Choh Jing Yi & Neo Li Xin & Joanne Ching Yin Xuan & Lim Yu Han

6.0 Design & Implementation:

This section explains how the memory allocation simulator is built step-by-step. The goal is to design a program that is simple and easily understand showing how different memory allocation strategies work. The entire implementation is done in Java.

6.1 System Flowchart

Steps in the Flowchart:



Start simulation: Run Memory Allocation Simulator code

1. Ask user for memory type: Choose memory type (1 = Fixed, 2 = Dynamic)
2. Input memory algorithm: Choose memory algorithm (1 = Next Fit, 2 = Worst Fit)
3. Input memory block: Enter the number of memory blocks and their sizes
4. Input number of processes: Enter the number of processes and their sizes
5. Run algorithm
6. Display algorithm result

6.2 Code Implementation

The program code is structured into one main class: MemoryAllocationSimulator. This class contains the main () method, handles all user inputs, and manages the simulation logic.

```
public class MemoryAllocationSimulator {  
    Run | Debug  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

User Input Section

This section prompts the user to choose the type of memory management (Fixed or Dynamic), the algorithm (Next Fit or Worst Fit), and to input memory block sizes and process sizes.

```
System.out.println(x:"=== OS Memory Management Simulation ===");  
System.out.println(x:"Memory Type:");  
System.out.println(x:"1. Fixed Partitioning");  
System.out.println(x:"2. Dynamic Partitioning");  
System.out.print(s:"Choose (1/2): ");  
int memType = sc.nextInt();  
  
System.out.println(x:"\nAlgorithm:");  
System.out.println(x:"1. Next Fit");  
System.out.println(x:"2. Worst Fit");  
System.out.print(s:"Choose (1/2): ");  
int algo = sc.nextInt();  
  
System.out.print(s:"\nEnter number of memory blocks: ");  
int nBlocks = sc.nextInt();  
int[] blocks = new int[nBlocks];  
int[] originalBlocks = new int[nBlocks];  
  
for (int i = 0; i < nBlocks; i++) {  
    System.out.print("Enter size of block " + (i+1) + ": ");  
    blocks[i] = sc.nextInt();  
    originalBlocks[i] = blocks[i]; // keep original copy  
}  
  
System.out.print(s:"\nEnter number of processes: ");  
int nProcesses = sc.nextInt();  
int[] processes = new int[nProcesses];  
  
for (int i = 0; i < nProcesses; i++) {  
    System.out.print("Enter size of process " + (i + 1) + ": ");  
    processes[i] = sc.nextInt();  
}
```

Allocation Selection and Execution

Based on the user's input, this section selects and runs the appropriate memory allocation algorithm for either Fixed or Dynamic Partitioning.

```

if (memType == 1) { // Fixed
    boolean[] blockAllocated = new boolean[nBlocks];

    if (algo == 1)
        nextFitFixed(processes, blocks, allocation, blockAllocated);
    else
        worstFitFixed(processes, blocks, allocation, blockAllocated);

    printResult(processes, allocation, originalBlocks, blockAllocated, isDynamic:false);
} else { // Dynamic
    if (algo == 1)
        nextFitDynamic(processes, blocks, allocation);
    else
        worstFitDynamic(processes, blocks, allocation);

    printResult(processes, allocation, blocks, fixedStatus:null, isDynamic:true);
}

```

Four core algorithm methods are implemented:

- **nextFitFixed()** – Next Fit on Fixed Partitioning

Allocates memory using the Next Fit algorithm under Fixed Partitioning. Starts searching from the last allocated block and finds the next available one that fits the process size.

```

// Next Fit for Fixed
public static void nextFitFixed(int[] processes, int[] blocks, int[] allocation, boolean[] blockAllocated) {
    int lastIndex = 0;
    int n = blocks.length;
    for (int i = 0; i < processes.length; i++) {
        allocation[i] = -1;
        int count = 0;
        while (count < n) {
            int idx = (lastIndex + count) % n;
            if (!blockAllocated[idx] && blocks[idx] >= processes[i]) {
                allocation[i] = idx;
                blockAllocated[idx] = true;
                lastIndex = idx;
                break;
            }
            count++;
        }
    }
}

```

- **worstFitFixed()** – Worst Fit on Fixed Partitioning

Allocates memory using the Worst Fit algorithm under Fixed Partitioning. Always chooses the largest available block that can fit the process.

```
// Worst Fit for Fixed
public static void worstFitFixed(int[] processes, int[] blocks, int[] allocation, boolean[] blockAllocated) {
    for (int i = 0; i < processes.length; i++) {
        allocation[i] = -1;
        int worstIdx = -1;
        for (int j = 0; j < blocks.length; j++) {
            if (!blockAllocated[j] && blocks[j] >= processes[i]) {
                if (worstIdx == -1 || blocks[j] > blocks[worstIdx]) {
                    worstIdx = j;
                }
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blockAllocated[worstIdx] = true;
        }
    }
}
```

- **nextFitDynamic()** – Next Fit on Dynamic Partitioning

Implements Next Fit for Dynamic Partitioning. Like Fixed, it continues searching from the last successful allocation, but also shrinks the block size after allocating part of it to a process.

```
// Next Fit for Dynamic
public static void nextFitDynamic(int[] processes, int[] blocks, int[] allocation) {
    int lastIndex = 0;
    int n = blocks.length;
    for (int i = 0; i < processes.length; i++) {
        allocation[i] = -1;
        int count = 0;
        while (count < n) {
            int idx = (lastIndex + count) % n;
            if (blocks[idx] >= processes[i]) {
                allocation[i] = idx;
                blocks[idx] -= processes[i];
                lastIndex = idx;
                break;
            }
            count++;
        }
    }
}
```

- **worstFitDynamic()** – Worst Fit on Dynamic Partitioning

Implements Worst Fit for Dynamic Partitioning. It looks for the largest block that can fit the current process and then reduces the size of that block after allocation.

```
// Worst Fit for Dynamic
public static void worstFitDynamic(int[] processes, int[] blocks, int[] allocation) {
    for (int i = 0; i < processes.length; i++) {
        allocation[i] = -1;
        int worstIdx = -1;
        for (int j = 0; j < blocks.length; j++) {
            if (blocks[j] >= processes[i]) {
                if (worstIdx == -1 || blocks[j] > blocks[worstIdx]) {
                    worstIdx = j;
                }
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blocks[worstIdx] -= processes[i];
        }
    }
}
```

printResult() method used to display the outcome of the simulation in a clear and readable format.

```
public static void printResult(int[] processes, int[] allocation, int[] blocks, boolean[] fixedStatus, boolean isDynamic) {
    System.out.println(x:"\n===== Allocation Result =====");
    System.out.println(x:"Process No\tProcess Size\tBlock Allocated");
    for (int i = 0; i < processes.length; i++) {
        String result = (allocation[i] == -1) ? "Not Allocated" : "Block " + (allocation[i] + 1);
        System.out.printf(format:"%d\t%d\t%s\n", i + 1, processes[i], result);
    }

    System.out.println();
    if (isDynamic) {
        System.out.println(x:"Remaining Block Sizes (Dynamic):");
        for (int i = 0; i < blocks.length; i++) {
            System.out.println("Block " + (i + 1) + ": " + blocks[i]);
        }
    } else {
        System.out.println(x:"Block Allocation Status (Fixed):");
        for (int i = 0; i < blocks.length; i++) {
            System.out.println("Block " + (i + 1) + ": " + (fixedStatus[i] ? "Allocated" : "Free"));
        }
    }
}
```

7.0 Results and Discussion

7.1 Interface and Execution

The Java program that runs the simulation is console-based and communicates with the user directly. Users are prompted to choose between a variety of options when the software is launched, including the memory allocation algorithm (Next Fit or Worst Fit), the memory management type (Fixed or Dynamic), the number and size of memory blocks, and the number and size of processes.

The system shows memory fragmentation (if any), emphasises rejected processes, and visually shows which processes are assigned to which memory blocks when inputs are received. Without the need for a complex graphical user interface, this straightforward text-based interface guarantees accessibility and clarity for learners.

7.2 Output Analysis

The outputs show how each method performs and behaves when using different partitioning techniques:

- **Next Fit (Fixed):** Allocates memory linearly, beginning with the most recent block. Although this method is quick, it might miss good blocks that show up earlier in the memory space, which would exacerbate fragmentation.
- **Worst Fit (Fixed):** Selects the biggest block that each process can use. Due to the inefficient fragmentation of big blocks, this can lead to underutilisation of space in fixed partitioning.
- **Next Fit (Dynamic):** Reduces the block size after each allocation, improving speed and memory reuse. It might still, however, exclude more appropriate, smaller pieces, which would lead to external fragmentation.
- **Worst Fit (Dynamic):** Reduces exterior fragmentation by filling the biggest gaps, providing better space utilisation. But over time, picking the biggest block repeatedly could leave behind little, useless pieces.

The outcomes of the simulation support theoretical predictions and aid in the visualisation of ideas such as block reuse, allocation failures, and fragmentation.

7.3 Discussion

The difficulties listed in the problem statement (Section 3.0) are successfully handled by the simulation. With its real-time, step-by-step memory allocation visualisation, the tool helps close the knowledge gap between theory and practice. It helps students visualise abstract ideas like algorithm efficiency and internal and external fragmentation.

In certain situations, Next Fit compromises proper memory utilisation even though it offers speed gains. Worst Fit exhibits superior fragmentation management, particularly in dynamic memory circumstances, despite being slower. This illustrates how performance and memory efficiency are traded off in the real world. Thus, by allowing users to try out various tactics and observe the results, the simulation promotes deeper learning.

8.0 Future Work & Conclusion

8.1 Future Work

To enhance the simulation's capabilities, several improvements can be considered:

- **Graphical User Interface (GUI):** The simulation would become more user-friendly and visually appealing if a GUI-based version were created.
- **Additional Algorithms:** For a more thorough comparison, other memory allocation techniques such as Best Fit, First Fit, and Buddy System can be integrated.
- **Statistical Analysis:** Incorporate various performance indicators such as process rejection rates, fragmentation percentage, and allocation time.

8.2 Conclusion

Through a terminal application built in Java, this project effectively uses the Next Fit and Worst Fit algorithms to simulate both fixed and dynamic memory management. It is a teaching aid for visualising and comprehending the internal workings of memory allocation.

Although the current version succeeds in illustrating fragmentation and allocation behaviour, it is constrained by the absence of a graphical user interface and a wider variety of algorithms. Improvements in the future might greatly improve the instructional value and user experience.

In general, the simulation has demonstrated efficacy in tackling learning obstacles related to operating system memory management and provides a basis for additional investigation and advancement.

9.0 Acknowledgement

We would like to convey our heartfelt gratitude to our respected lecturer, Dr. Danlami Gabi, for his continuous support and competent guidance during the project's development. Her extensive knowledge and enthusiasm for teaching have been crucial in moulding our grasp of this complex subject. Dr. Danlami's consistent encouragement and constructive feedback inspired us to aim higher and substantially aided in the successful completion of our task. His commitment to our learning and development has not only improved our technical skills, but also deepened our appreciation for the practical applications of operating system concepts. This endeavour would not have been feasible without his unwavering support and inspirational mentoring. We are extremely appreciative for the opportunity to learn under his leadership. Thank you, Dr. Danlami, for being an inspiration and playing an important role in our academic career.

10.0 References

1. OS Simulator. (2023). Online memory management simulator. <http://www.ossimulator.com>
2. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.
3. Stallings, W. (2017). Operating systems: Internals and design principles (9th ed.). Pearson.
- Tanenbaum, A. S. (2014). Modern operating systems (4th ed.). Pearson.

11.0 Presentation Link

<https://youtu.be/mBvRJ3mCUe0?si=FT5vQatse2H1wo8T>